

ПЕРЕТВОРЕННЯ ПРОГРАМНОЇ МОДЕЛІ У ЦІЛЬОВУ ПРОГРАМУ

Розробка безпомилкових і надійних програм стала нагальною задачею для сучасних технологій програм, на основі яких будуються власно програми. Програмні моделі спочатку повинні бути алгоритмічно описані і на основі опису створюватись програмні моделі. Для такого опису широко застосовується предикати із застосуванням темпоральної логіки. Створення таких програм особливо важливо у галузі управління в різних сферах: транспорті, виробничих процесах, військовій справі, тощо. Дослідження і розробки у цьому напрямку є досить нагальними і актуальними. Через помилки у програмному забезпеченні виникали катастрофи в різних системах управління. Варто тільки згадати катастрофи з літаками BOING 737 в Індонезії (жовтень 2018 р.) і Ефіопії (березень 2019 р.) через помилки у програмах управління, а помилка у програмі управління деяких моделей авіалайнерів Airbus A350 необхідно перезавантажувати кожні 149 годин, щоб запобігти часткову або повну втрату функціональності окремих систем [1]. У разі автоматизованого перетворення моделей у програми виключаються помилки, підвищується надійність програм. Для такого перетворення в програмній моделі треба виділити основні програмні блоки: лінійні, розгалуження і цикли різних типів, які треба перетворити у проміжне подання і далі в оператори цільової мови програмування. Деякі специфічні процедури, які мають використовуватись окремими блоками, можуть бути створені окремо або також на основі побудови їх моделей.

Спроби доказу коректності програм ведуться давно. Перші з них базувались із застосуванням припущень і аксіом, розроблених Хоаром [2] або за допомогою специфічних обчислень, запропонованих Мілнером і розвинутих його послідовниками [3] виявились доволі складними та громіздкими і тому не знайшли практичного застосування. Технологія структурного програмування дозволяє впорядковувати процес програмування, але не дозволяє уникнути помилок. Подальші технології [4] дозволяли створювати лише загальну структуру програм, але не дозволяли створювати безпомилкові власно структурні блоки. Останні розробки пов'язані із розробкою моделей програм, які можна верифікувати і на їх основі розробляти власно програми [5]. Така технологія за основу взяла автоматну модель на основі структури Крипке і темпоральної логіки [5]. До недоліку такого підходу є непрямий опис моделі, а тільки окремих його гілок за допомогою предикатів, а сама модель має створюватись за допомогою спеціальної програми. Далі така модель перевіряється за допомогою спеціальної програми-верифікатора на предмет її відповідності усім специфікаціям і обмеженням, які визначені при описі моделей. Це створює певні труднощі при побудові моделей, до того ж згадані програми є власністю фірми-виробника.

Тому метою даної статті є опис технології прямого опису моделі програми саме так, як це раніше виконувалось групою алгоритмістів та подальшого створення на її основі спочатку моделі програм і далі самі програми. Раніше група програмістів на основі опису цільової програми розробляла власно програму. Але за відсутності повного формального опису моделей програм, при створенні програм виникають деякі пропуски обробки за певних умов, що і є основною причиною виникнення помилок. Як вже було зазначено модель програми створюється на основі її безпосереднього опису за допомогою темпоральної логіки і предикатів [6]. У такому випадку відпадає необхідність будувати модель програми з окремих частин та наступної її верифікації спеціальними програмами, як це передбачено технологією MODEL CHECKING [5]. При цьому в описі моделі треба описати обробку усіх подій, які можуть виникнути. Такі події набагато простіше передбачити на етапі моделювання у процесі опису власно моделі.

Цілком природно подавати опис моделі програми у вигляді послідовності взаємопов'язаних предикатів. При цьому декларативна частина предикату має описувати певні дії, а логічна частина повинна описувати умови для їх виконання. Математичною

моделлю, яка прийнята для технології прямого опису моделі програми, є структура Крипке, що являє собою певну форму подання автомату орієнтовану на застосування предикатів і являє собою наступний кортеж: $M = (S, S_0, R, AP, L)$,

де S – набір станів моделі;

S_0 – набір початкових станів моделі;

$R = S \times S$ – визначає реальні зв'язки між станами;

AP – кінцевий набір предикатів;

$L = 2^{AP}$ – визначає загальну можливу сукупність предикатів.

У результаті опису моделі програми створюється модель у вигляді недетермінованого скінченного автомату. У подальшому ця модель буде використовуватись для розробки власно цільової програми.

Обробка моделі програми полягає у виявленні основних блоків, а саме лінійні блоки, блоки розгалуження і блоки циклів. Лінійні блоки визначають оператори присвоєння і не викликають жодних труднощів. Присвоєння вибираються із змістовних частин предикатів. Щодо розгалужень, то існують прості розгалуження за однією умовою та розгалуження за кількома умовами. Причому у разі кількох умов, якщо розгалуження здійснюються за значенням одного параметру, то маємо прообраз оператора типу CASE, а коли розгалуження здійснюються за значеннями кількох параметрів, то маємо прообраз багато вкладеного оператора IF. Графічно види розгалужень наведені нижче на рис. 1.

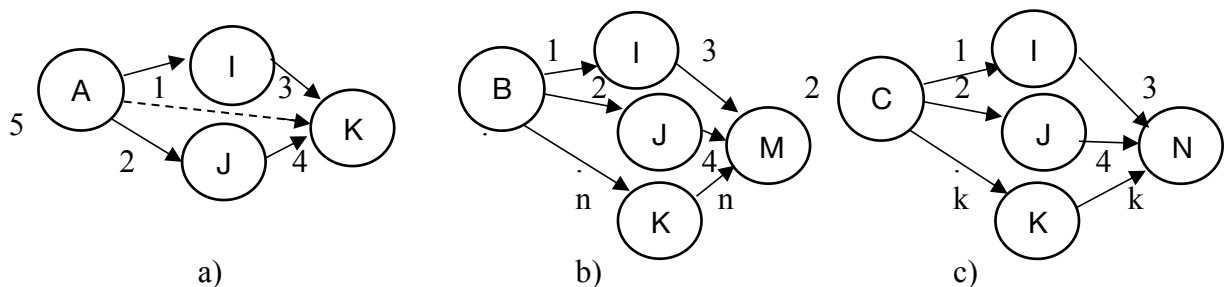


Рис. 1. Види розгалужень

На рисунку 1 а) представлений варіант класичного розгалуження. За гілкою 1 розгалуження здійснюється за певною умовою (аналог IF), за гілкою 2 розгалуження здійснюється за протилежною умовою (аналог else). В обох випадках обидві гілки розгалуження входять у спільний стан К (зв'язки 3 та 4). Для цього виду розгалуження можливий варіант відсутності розгалуження за протилежною умовою (аналог варіанту else) і тоді маємо зв'язок за гілкою 5 (показано пунктиром). На рисунку 1 б) представлено розгалуження за різними значеннями одного параметру від 1 до n (аналог CASE), а на рисунку 1 c) представлено розгалуження за різними параметрами від 1 до k (аналог if ... elseif...).

За допомогою предикатів [5] опис розгалуження 1 а) можна представити таким чином: $(\{COND_1\}[branch_1] \times \{true\}[operators] \text{ OR } \{NOT\} COND_1\}[branch_2] \times \{true\}[operators]) \times \{true\}[operators]$. Тобто маємо розгалуження за прямою умовою ($COND_1$) і за протилежною умовою ($NOT\} COND_1$) і виконання дій за цими умовами ($operators$). Розгалуження виду 1 б) можна представити так $\{PAR=X_1\}[branch_1] \times \{true\}[operators] \text{ OR } \{PAR=X_2\}[branch_2] \times \{true\}[operators] \text{ OR } \dots \{PAR=X_n\}[branch_n] \times \{true\}[operators]$, тобто розгалуження за кількома значеннями параметру PAR від X_1 до X_n і виконання відповідних дій за цими умовами. Опис розгалуження виду 1 c) має такий вигляд $\{COND_1\}[branch_1] \times \{true\}[operators] \text{ OR } \{COND_2\}[branch_2] \times \{true\}[operators] \text{ OR } \dots \{COND_k\}[branch_k] \times \{true\}[operators]$. У даному випадку маємо розгалуження за різними умовами і виконання дій за цими умовами.

При розпізнаванні циклів треба спочатку визначити їх тип. Для циклу за перерахунком треба визначити кількість повторів та власно тіло циклу. За допомогою предикатів цей

цикл описується таким чином: $\{\text{true}\}[n=\text{vol}] \times \{n < 0\}[\text{cycle}]$. У такому випадку цикл виконується n разів. У разі циклу з передумовою (аналог DO WHILE) треба визначити умову циклу та його тіло. За допомогою предикатів цей цикл описується так: $\{\text{COND}\}[\text{cycle}]$, тобто цикл виконується якщо виконується умова COND. Якщо маємо цикл з післяумовою (аналог UNTIL), то спочатку виконується тіло циклу, а потім перевіряється умова його повтору. За допомогою предикатів цей цикл описується так: $\{\text{true}\}[\text{cycle}] \times \{\text{COND}\}[\text{cycle}]$. У кожному випадку позначка cycle означає тіло циклу у кожному конкретному випадку. Позначка COND визначає умову повтору циклу. Як бачимо цикл з післяумовою по меншій мірі виконується один раз, що і відображено при описі. Графічно усі зазначені види циклів у вигляді фрагментів автоматної моделі наведені нижче на рисунку 2.

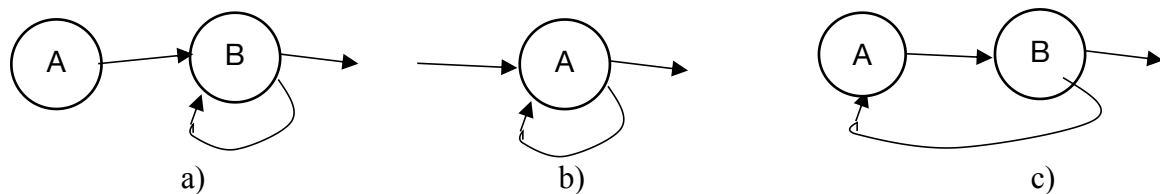


Рис. 2. Зображення циклів

Рисунок 2 а) показує тип циклу за перерахунком. У стані А відбувається підготовка до циклу, зокрема встановлюється кількість виконання циклів, а у стані В цикл буде виконуватись доти, доки кількість повторів не закінчиться. На рисунку 2 б) показано зображення циклу з передумовою. У стані А тіло циклу буде повторно виконуватись доти, доки виконується передумова. Рисунок 2 с) відображає цикл з післяумовою. У стані А виконується тіло циклу і далі тіло циклу виконуватиметься доти, доки у кінці циклу (стан В) виконуватиметься умова повторного виконання. В усіх трьох випадках вихід із циклу буде виконуватись, коли пересте діяти умова циклу.

Якщо передбачається використання певних програмних заготовок у вигляді процедур, які можна створити, перевірити і далі застосувати окремо за допомогою даної технології. Умовні та виконавчі частини предикатів визначаються з моделі програми у вигляді спеціальної бази даних [6], зокрема з опису у рядках невизначеної довжини. Ці типи даних мають оброблятися за допомогою синтаксичного аналізу, в результаті якого формується внутрішнє подання програми, яке у подальшому може бути перетворено на потрібну цільову мову програмування.

Кожен вид з означених типових програмних блоків у проміжному поданні може бути позначений певним символом, за яким мають бути розташовані відповідні дані. Так, для лінійного блоку це може бути конкретний вираз або кілька виразів, наприклад EX, ($\langle \text{вираз} \rangle$, $\langle \text{змінна} \rangle$). У подальшому таке подання, яке відображає оператор присвоєння для об'явленої або тимчасової змінної, при його перетворенні на цільову мову програмування на звичайні оператори присвоєння. Умовні блоки розглянутих типів можна позначити, наприклад наступним чином. Для звичайного розгалуження внутрішнє подання можна позначити наступним чином IF1, ($\langle \text{логічний вираз}_1 \rangle$, $\langle \text{дії}_1 \rangle$ [$\langle \text{логічний вираз}_2 \rangle$], $\langle \text{дії}_2 \rangle$), $\langle \text{дії}_3 \rangle$. Квадратні дужки вказують на необов'язковість конструкції (аналог else). Тобто за прямою умовою мають виконуватись $\langle \text{дії}_1 \rangle$, за протилежною умовою мають виконуватись $\langle \text{дії}_2 \rangle$, після чого мають виконуватись спільні дії $\langle \text{дії}_3 \rangle$. Для багаторівневого розгалуження типу CASE внутрішнє подання можна подати таким чином IF2 $\langle \text{PAR}=\text{VOL}_1 \rangle$, $\langle \text{дії}_1 \rangle$; $\langle \text{PAR}=\text{VOL}_2 \rangle$, $\langle \text{дії}_2 \rangle$; ... $\langle \text{PAR}=\text{VOL}_n \rangle$, $\langle \text{дії}_n \rangle$. Таким чином, для кожного конкретного значення деякого параметра виконуються певні дії. Для багаторівневого розгалуження за різними умовами внутрішнє подання можна подати так IF3 $\langle \text{COND}_1 \rangle$, $\langle \text{дії}_1 \rangle$; $\langle \text{COND}_2 \rangle$, $\langle \text{дії}_2 \rangle$; ... $\langle \text{COND}_k \rangle$, $\langle \text{дії}_k \rangle$. Для циклів з перерахунком внутрішнє подання може мати наступний вигляд DO1 $\langle \text{vol} \rangle$, $\langle \text{дії} \rangle$. Тут vol визначає

кількість циклічного виконання дій <ді>. У разі циклу з передумовою внутрішнє подання можна подати так: DO2 <COND>, <ді>, тобто вказується, що означені дії мають циклічно виконуватись якщо виконується умова <COND>. Для циклу з післяумовою внутрішнє подання може мати такий вигляд: DO3 <COND>, <ді>. На відміну від попереднього подання циклу <ді> обов'язково один раз має виконуватись, а далі цикл має виконуватись як попередній вид циклу. Нестандартні процедури, які можуть використовуватись у майбутній програмі, можуть створюватись за описаною технологією окремо або разом з основною програмою. Маючи внутрішнє подання майбутньої програми, з'являється можливість перетворити це подання у програму на цільовій мові програмування. Таким чином створення моделі програми за допомогою предикатів і темпоральної логіки дозволяє отримати внутрішнє подання програми, яке складається з основних програмних блоків. При цьому для перетворення внутрішнього подання в оператори цільової мови програмування у внутрішньому поданні формуються усі необхідні данні.

Опис моделі програми породжує базу даних, структура якої представлена нижче. Відношення MAIN описує поточний стан моделі, відношення CONNECT вказує на можливі розгалуження, а відношення SEM являє собою семафорний примітив.

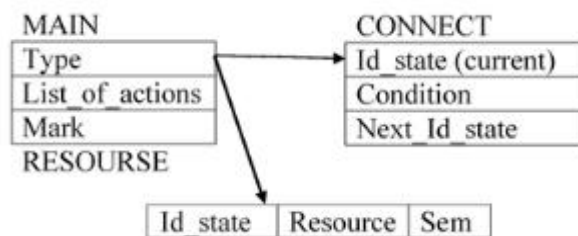


Рис. 3. Структура бази даних

Технологія створення моделі програми за допомогою опису моделі із застосуванням предикатів і темпоральної логіки дозволяє на етапі опису моделі програми і наступної її побудови врахувати всі специфікації, обмеження і умови, які виникають у процесі виконання програми на етапі моделювання і зреагувати на всі ці умови. Оскільки всі умови, які виникають в моделі максимально враховуються, то виключається поява помилок і перехід у мертвий код. На етапі моделювання можуть бути виявлені і усунуті всі можливі недоліки і помилки в моделі програми. При подальшому розвитку даної технології у разі автоматизації перетворення опису моделі програми власно у модель і далі у внутрішнє подання (у вигляді шаблонів) дозволяє створювати надійні безпомилкові програми. До того ж внутрішнє подання програми можна перетворити у програму на потрібній цільовій мові програмування. Але процес перетворення внутрішнього подання програми, який вже достатньо повно відпрацьований у компіляторах, також може бути автоматизований, що відкриває широкі перспективи у розвитку технології створення безпомилкових і надійних програм.

Посилання

1. [https://internetua.com/oshibka-v-po-airbus-a350-vynujdaet-perezagrujat-sistemy-samoletov-kajdye-149-csasov-airbus A-350 – перезавантаження програми управління через 149 годин](https://internetua.com/oshibka-v-po-airbus-a350-vynujdaet-perezagrujat-sistemy-samoletov-kajdye-149-csasov-airbus-A-350-perezavantazhenня-programи-управління-через-149-годин).
2. Hoare, C. A. R. Communicating sequential process. Prentice Hall International Publ., 2022. 260 p. Available at: <http://www.usinghcspp.com/capbook.pdf>.
3. Johannes Aman Pohjola. Foundation of Concurrency. The Calculus of Communicating Systems. UNSW Term 2 2022. 2. 68 p. Available at: <https://cgi.cse.unsw.edu.au/~cs3151/22T2/Week%2008/Wednesday%20Slides.pdf>.

4. The Unified Modeling Language User Guide, Second Edition. With James Rumbaugh and Ivar Jacobson. Addison-Wesley. 2005. [ISBN 978-0-321-26797-9](#).
5. Clarke, E. M., Granberg, O, Kroening, D., Peled, D. & Veith, H. Model Checking. Second Edition. MIT Press Publ., 2018. 424 p. ISBN 0262349450. Available at: <https://books.google.com.ua/books?Id=qJI8DwAAQBAJ>.
6. Sergii HOLUB, Volodymyr SALAPATOV, Vadym NEMCHENKO. REPRESENTATION OF THE PROGRAM MODEL USING PREDICATES. Radioelectronic and Computer Systems, 2024, No. 1(109), p. 6-15. Kharkiv, doi: 10:32620/reks.2024.1.01, ISSN 2663-2012.